

Lab 6.1: Introduction to OSCAL

OSCAL is NIST's machine-readable format for controls, profiles, components, system security plans, and assessments. The point is not the format. The point is that an assessor can traverse from a catalog, to a profile, to a component, to an evidence URI, and verify your control **without ever talking to you**. The audit becomes a graph traversal.

This lab writes the smallest useful piece of that graph, and it **generates** the control mappings from the policies that enforce them rather than retyping them by hand.

For reading. Code lines wrap to fit the page; copy code from the repository, not the PDF.

GRC Engineering Club

grcengclub.com · based on GRCEngClub/cgep-labs

FROM CHECKBOX GRC TO ENGINEERED SYSTEMS

This PDF is for reading. Long code lines wrap to fit the page, so copy commands and code from `docs/06_01_introduction_to_oscal.md` in your clone, not from the PDF.

Learning objectives

- Author a valid Component Definition and a Profile, validated by `trestle`.
- Use `implementation-status` honestly: `implemented`, `partial`, `inherited`.
- Generate `implemented-requirement` entries from Lab 3.3's `policy-catalog.json`.
- **Verify that every evidence URI resolves**, because OSCAL will not check that for you.

Prerequisites

- Run these from inside the devcontainer if you set one up in Lab 0.1: that is where the toolchain and your cloud logins live. `source cgep.env` first, in every new shell.
- Python `>= 3.10`, `pip install compliance-trestle`.
- Lab 2.3 (v2) and/or Lab 2.4 module on disk. We describe them in OSCAL.
- Lab 2.5 vault exists, with at least one bundle, so evidence URIs resolve.
- Lab 3.3's `evidence/lab-3-3/policy-catalog.json`.

Estimated time & cost

- 75 to 90 minutes.
- Free. No cloud calls beyond fetching the NIST catalog and one `head-object` per evidence URI.

The five OSCAL models

Model	Describes	Built here
Catalog	A library of controls (NIST 800-53 Rev 5)	No. We link to NIST's.
Profile	A subset selected from one or more catalogs	Yes, minimal.
Component Definition	How a component implements specific controls	Yes, the centerpiece.
SSP	A whole system's controls and components	Capstone stretch goal.
Assessment Plan / Results	What the assessor planned and found	Out of scope.

Architecture

```
Terraform module      OSCAL      Assessor
-----
compliant-s3/          --describes--> component-definition.json
main.tf, ...           control-implementations
                        source: NIST 800-53 rev5
                        implemented-requirements
                        sc-8, sc-12, sc-13, sc-28,
                        ac-3, ac-6, au-3, au-9,
                        au-11, cm-6, cm-8
                        props: terraform-resource
                        links: rel=evidence -> s3://...?versionId=
                        |
policy-catalog.json    --generates--> |
(Lab 3.3)              v              |
                        profile.json    follows href into the vault
                        (selects those ids) runs verify-evidence.sh
                                           sees CHAIN INTACT
```

Step-by-step walkthrough

Step 1 Initialize a trestle workspace

```
pip install compliance-trestle
mkdir lab-6-1 && cd lab-6-1
trestle init
trestle version    # note the OSCAL version it pins
```

Write that version down. Catalog, profile, and component must all share an `oscal-version`, and mixing them is the single most common validation failure.

Step 2 Generate the requirements from the policy catalog

This is the v2 change, and it is the whole idea.

The obvious approach is to hand-type each `implemented-requirement`, having already typed the same control ID into the Rego metadata in Lab 3.3. That is two copies of one mapping, in two languages, drifting apart the moment either changes.

Generate one from the other instead:

```
#!/usr/bin/env python3
# scripts/gen-oscal-requirements.py
"""Build OSCAL implemented-requirements from the Lab 3.3 policy catalog.

Each Rego policy declares its control_id in a # METADATA block. That is the
single source of truth; this script projects it into OSCAL so the mapping is
authored once and never retyped.
"""
import json, sys, uuid, pathlib

catalog_path = pathlib.Path(sys.argv[1])          # evidence/lab-3-3/policy-catalog.json
evidence_uri = sys.argv[2]                       # s3://VAULT/runs/ID/bundle.tar.gz?versionId=...

# Controls this component satisfies by means OTHER than a Rego policy, plus
# any whose status is not a plain "implemented". Everything here is a claim
# you must be able to defend out loud.
EXTRA = {
    "sc-12": ("implemented", "aws_kms_key.bucket with enable_key_rotation = true."),
    "sc-13": ("implemented", "SSE-KMS with a customer-managed key."),
    "ac-6":  ("implemented", "aws_s3_bucket_ownership_controls = BucketOwnerEnforced; ACLs disabled."),
    '
    "au-11": ("implemented", "aws_s3_bucket_lifecycle_configuration expires logs at 90 days."),
    "cm-8":  ("implemented", "Provider default_tags applies ComplianceScope to every taggable
resource."),
    "au-6":  ("partial",      "Athena queries authored in Lab 5.2. Scheduled review not yet
automated."),
    }

TERRAFORM_RESOURCE = {
    "sc-8":  "aws_s3_bucket_policy.primary",
    "sc-12": "aws_kms_key.bucket",
    "sc-13": "aws_s3_bucket_server_side_encryption_configuration.primary",
    "sc-28": "aws_s3_bucket_server_side_encryption_configuration.primary",
    "ac-3":  "aws_s3_bucket_public_access_block.primary",
    "ac-6":  "aws_s3_bucket_ownership_controls.primary",
    "au-3":  "aws_s3_bucket_logging.primary",
    "au-9":  "aws_s3_bucket_versioning.log",
    "au-11": "aws_s3_bucket_lifecycle_configuration.log",
    "cm-6":  "provider.aws.default_tags",
    "cm-8":  "provider.aws.default_tags",
    }

policies = json.loads(catalog_path.read_text())
seen, reqs = set(), []

def add(control_id, description, status, policy_pkg=None):
    if control_id in seen:
        return
    seen.add(control_id)
    props = [{"name": "implementation-status", "value": status}]
    if control_id in TERRAFORM_RESOURCE:
        props.append({"name": "terraform-resource", "value": TERRAFORM_RESOURCE[control_id]})
    if policy_pkg:
        props.append({"name": "enforced-by-policy", "value": policy_pkg})
    reqs.append({
        "uuid": str(uuid.uuid4()),
```

```

        "control-id": control_id,
        "description": description,
        "props": props,
        "links": [{
            "rel": "evidence",
            "href": evidence_uri,
            "text": "Signed pipeline bundle containing terraform plan.json.",
        }],
    })

for p in policies:
    cid = (p.get("control") or "").lower()
    if not cid:
        continue
    add(cid,
        f"{p['title']} Enforced in the pipeline by Rego policy {p['package']}, "
        f"severity {p['severity']}. Remediation: {p.get('remediation', 'see policy')}",
        "implemented",
        p["package"])

for cid, (status, desc) in EXTRA.items():
    add(cid, desc, status)

json.dump(sorted(reqs, key=lambda r: r["control-id"]), sys.stdout, indent=2)

```

```

python3 scripts/gen-oscal-requirements.py \
    evidence/lab-3-3/policy-catalog.json \
    "s3://YOUR_VAULT/runs/RUN_ID/bundle.tar.gz?versionId=VERSION_ID" \
    > /tmp/requirements.json

```

Notice what the `EXTRA` dictionary is really for. It is the list of controls you claim on grounds **other** than an automated policy check, and every entry is something an assessor can ask you to justify. Keeping it small and explicit is the discipline. A component whose every claim came from a passing policy is a strong component; one whose claims are mostly hand-written prose is a document.

Step 3 The component definition

```
trestle create -t component-definition -o compliant-s3-v2 -x json
```

Then replace the skeleton. The shape, with the generated requirements spliced in:

```

{
  "component-definition": {
    "uuid": "GENERATED-UUID-V4",
    "metadata": {
      "title": "compliant-s3 module v2",
      "last-modified": "2026-08-17T18:00:00.000000+00:00",
      "version": "2.0.0",
      "oscal-version": "1.2.1",
    },
  },
}

```

```

    "parties": [
      {
        "uuid": "PARTY-UUID-V4",
        "type": "organization",
        "name": "Your organization"
      },
      {
        "uuid": "CSP-UUID-V4",
        "type": "organization",
        "name": "Amazon Web Services"
      }
    ],
    "components": [
      {
        "uuid": "COMPONENT-UUID-V4",
        "type": "software",
        "title": "compliant-s3",
        "description": "Reusable Terraform pattern for an AWS S3 primary bucket plus a dedicated
access-log bucket. Enforces SSE-KMS with a customer-managed key, TLS-only access, versioning on both
buckets, full public access block, ACLs disabled, lifecycle retention, access logging, and required
compliance tags.",
        "purpose": "Provide a compliant-by-default S3 primitive that any team can adopt with three
lines of consumer Terraform.",
        "responsible-roles": [
          { "role-id": "provider", "party-uuids": ["PARTY-UUID-V4"] }
        ],
        "control-implementations": [
          {
            "uuid": "CI-UUID-V4",
            "source": "https://raw.githubusercontent.com/usnistgov/oscal-content/v1.2.1/nist.gov/
SP800-53/rev5/json/NIST_SP-800-53_rev5_catalog.json",
            "description": "NIST 800-53 Rev 5 controls satisfied by this module.",
            "implemented-requirements": [ "<<< generated by gen-oscal-requirements.py >>>" ]
          }
        ]
      }
    ]
  }
}

```

Splice them in:

```

jq --slurpfile reqs /tmp/requirements.json \
'["component-definition"].components[0]["control-implementations"][0]["implemented-requirements"] =
$reqs[0]' \
  component-definitions/compliant-s3-v2/component-definition.json > /tmp/cd.json \
  && mv /tmp/cd.json component-definitions/compliant-s3-v2/component-definition.json

```

Anchor the catalog source to a tag, not main. NIST moves that branch, so an assessor opening your component a year from now would get whatever is current rather than the catalog you actually assessed against. Same argument as pinning a provider version, applied to a control catalog. Check the tag exists: several plausible-looking ones return 404.

Generate UUIDs properly. OSCAL requires v4 (xxxxxxxx-xxxx-4xxx-yxxx-... where y is 8/9/a/b). Run `python3 -c "import uuid; print(uuid.uuid4())"`. `trestle validate` rejects the wrong shape with a regex error that tells you nothing useful.

Step 4 Inherited controls, done honestly

If you are also describing Lab 2.4's GCP module, SC-8 is satisfied by the platform rather than your code. OSCAL has a way to say that, and using it is the difference between an accurate component and an overclaim:

```
{
  "uuid": "REQ-UUID-V4",
  "control-id": "sc-8",
  "description": "Transmission confidentiality is provided by the Google Cloud Storage API, which
accepts only TLS connections. No module-level control is implemented; this control is inherited from
the cloud service provider.",
  "props": [
    { "name": "implementation-status", "value": "inherited" },
    { "name": "leveraged-authorization", "value": "Google Cloud Platform" }
  ],
  "responsible-roles": [
    { "role-id": "cloud-service-provider", "party-uuids": ["CSP-UUID-V4"] }
  ]
}
```

Three statuses, three meanings, and students conflate them constantly:

Status	Means	The question it invites
implemented	Your code enforces it	"Show me the line."
partial	Some of it, and you can say which part	"What is missing, and when?"
inherited	The provider enforces it; you rely on that	"Where is their authorization?"

AU-6 in this component is `partial` on purpose. Lab 5.2 authored the Athena queries; nothing schedules them. Claiming `implemented` there would be claiming a control on the strength of having built the tooling for it.

Step 5 Validate

```
trestle validate -f component-definitions/compliant-s3-v2/component-definition.json
```

```
VALID: Model ../component-definition.json passed the Validator
```

Step 6 The profile

```
trestle create -t profile -o cge-p-minimum -x json
```

```
{
  "profile": {
    "uuid": "PROFILE-UUID-V4",
    "metadata": {
      "title": "CGE-P minimum control selection",
      "last-modified": "2026-08-17T18:00:00.000000+00:00",
      "version": "2.0.0",
      "oscal-version": "1.2.1"
    },
    "imports": [
      {
        "href": "https://raw.githubusercontent.com/usnistgov/oscal-content/v1.2.1/nist.gov/SP800-53/rev5/json/NIST_SP-800-53_rev5_catalog.json",
        "include-controls": [
          {
            "with-ids": [
              "ac-3", "ac-6", "au-3", "au-6", "au-9", "au-11",
              "cm-6", "cm-8", "sc-8", "sc-12", "sc-13", "sc-28"
            ]
          }
        ]
      }
    ],
    "merge": { "as-is": true }
  }
}
```

```
trestle validate -f profiles/cge-p-minimum/profile.json
trestle profile-resolve -n cge-p-minimum -o cge-p-minimum-resolved
```

The profile and the component must agree: **every control in the component must appear in the profile**. Step 7 checks that mechanically, because doing it by eye stops working around control number six.

Step 7 Verify the graph, since OSCAL will not

This usually shows up as a footnote: OSCAL does not validate that hrefs resolve, so a broken URI is silently a useless attestation.

It belongs in the lab rather than the footnotes. A component definition whose evidence links 404 passes `trestle validate` cleanly. It is a perfectly valid document describing nothing.

```
#!/usr/bin/env bash
# scripts/verify-oscal-graph.sh <component-definition.json> <profile.json>
set -euo pipefail

CD="${1:?usage: verify-oscal-graph.sh <component-def> <profile>}"
PROFILE="${2:?}"
PROFILE_ARG="${AWS_PROFILE:+--profile $AWS_PROFILE}"
FAIL=0

echo "=== 1. Every evidence href resolves ==="
while IFS= read -r href; do
  case "$href" in
    s3://*)
      rest="${href#s3://}"
      bucket="${rest%/*}"
      keypart="${rest#*/}"
      key="${keypart%%\?*}"
      version=""
      [[ "$keypart" == *"versionId="* ]] && version="--version-id ${keypart##*versionId=}"
      if aws $PROFILE_ARG s3api head-object --bucket "$bucket" --key "$key" $version >/dev/null 2>&1;
      then
        echo " OK      $href"
      else
        echo " BROKEN  $href"; FAIL=1
      fi
      ;;
    http://*|https://*)
      if curl -fsSL -o /dev/null --max-time 20 "$href"; then
        echo " OK      $href"
      else
        echo " BROKEN  $href"; FAIL=1
      fi
      ;;
    *) echo " SKIP    $href (unrecognized scheme)" ;;
  esac
done < <(jq -r '.. | objects | select(.rel? == "evidence") | .href' "$CD" | sort -u)

echo "=== 2. Catalog source resolves and is version-pinned ==="
SRC=$(jq -r '.. | objects | select(has("source")) | .source' "$CD" | head -1)
curl -fsSL -o /dev/null --max-time 30 "$SRC" \
  && echo " OK      $SRC" || { echo " BROKEN  $SRC"; FAIL=1; }
case "$SRC" in
  */main/*) echo " WARN    catalog source tracks 'main'; pin to a tag"; ;;
esac

echo "=== 3. Every component control appears in the profile ==="
comm -23 \
  <(jq -r '.. | objects | select(has("control-id")) | .["control-id"]' "$CD" | sort -u) \
  <(jq -r '.profile.imports[].include-controls[].with-ids[]' "$PROFILE" | sort -u) \
  | while read -r missing; do
    echo " MISSING from profile: $missing"; FAIL=1
  done
```

```

echo "=== 4. No placeholder UUIDs survived ==="
if grep -qE '(GENERATED|PARTY|COMPONENT|CI|REQ|PROFILE|CSP)-UUID-V4' "$CD" "$PROFILE"; then
    echo "  FAIL: placeholder UUIDs still present"; FAIL=1
else
    echo "  OK"
fi

[[ $FAIL -eq 0 ]] && echo && echo "OSCAL GRAPH INTACT" || { echo; echo "OSCAL GRAPH BROKEN"; exit 1; }

```

Run it, and put it in CI next to `trestle validate`. Validation proves the document is well-formed. This proves it is **true**.

Step 8 Demonstrate the traversal

Pick `sc-28` in the component. Follow `links[rel=evidence].href` into the vault. Run Lab 4.4's script:

```
EVIDENCE_VAULT=YOUR_VAULT bash scripts/verify-evidence.sh RUN_ID
```

`CHAIN INTACT`.

The catalog, the profile, the implementation statement, the evidence URI, and the signed bundle are now one connected graph. An assessor reading the OSCAL verifies your control without you in the room, which is the entire thesis of this course stated as a file format.

Verification

- `trestle validate` returns `VALID` for the component definition **and** the profile.
- `trestle profile-resolve` produces a resolved profile.
- `scripts/verify-oscal-graph.sh` prints `OSCAL GRAPH INTACT`.
- At least one evidence URI resolves to a real signed object, by version.
- `implementation-status` values are honest: nothing marked `implemented` that you cannot demonstrate.

Portfolio submission checklist

- ☐ `oscal/components/<your-component>.json`, `trestle-validated`.
- ☐ `oscal/profiles/cge-p-minimum.json`, `validated`.
- ☐ `scripts/gen-oscal-requirements.py` and `scripts/verify-oscal-graph.sh` committed.
- ☐ `evidence/lab-6-1/trestle-validate.txt` and `evidence/lab-6-1/graph-verify.txt`.
- ☐ `oscal/README.md` naming which module each component describes, where evidence lives, and **which controls are partial or inherited and why**.

Troubleshooting

- **string does not match regex on UUIDs.** OSCAL requires v4. Generate them; do not hand-write.
- **Missing required fields.** `trestle describe -t component-definition -n <name>` prints the schema requirements verbosely.
- **Evidence URIs that do not resolve.** OSCAL does not check. That is what Step 7 is for.
- **Catalog import fails.** NIST's URLs move. Anchor to a tag rather than `main`.
- **Version mismatch.** Catalog, profile, and component must share `oscal-version`. Check `trestle version`.
- **`jq --slurpfile` produced a nested array.** `$reqs[0]` unwraps it. Without the index you get `[[...]]` and `trestle` fails on the type.

Cleanup

OSCAL is JSON in your repo. Nothing in the cloud. Commit and move on.

How this feeds the capstone

```
oscal/  
  components/YOUR_COMPONENT.json      # what you built  
  profiles/cge-p-minimum.json         # the controls you claim
```

The evidence hrefs point at the latest signed bundle in your vault, written by the Lab 4.3 and 4.4 pipeline. The chain ends in the vault.

Two things will distinguish your submission. **`verify-oscal-graph.sh` in CI**, because the capstone brief warns that "an OSCAL file that doesn't actually describe your system is worse than no OSCAL file," and this script is the mechanical answer to that. And **honest `implementation-status` values**, because the graders check whether the OSCAL describes the system you built, and a component claiming twelve `implemented` controls where two are really `partial` is the exact failure the brief calls out as copy-paste OSCAL.

When you retarget from NIST 800-53 to HIPAA, SOC 2, or CMMC for the capstone, change the `control_id` in each Rego metadata block and the `source` catalog URI, then re-run the generator. The mapping was authored once. That is why it was worth generating.

Revision history

v2 (current)

- **`implemented-requirement`** entries are generated from the Lab 3.3 policy catalog rather than hand-typed, so each control mapping is authored once in the Rego annotation that enforces it.

- Added `verify-oscal-graph.sh`: checks that every evidence href resolves, the catalog source is reachable and version-pinned, every component control appears in the profile, and no placeholder UUIDs survive. `trestle validate` proves the document is well-formed; this proves it is true.
- `implementation-status` is used honestly, including `partial` and `inherited`, with the distinction spelled out.
- `oscal-version` and the catalog tag aligned to the version `trestle` targets, and the guidance now says to confirm the tag exists.
- Control coverage grew from four to fourteen as the Chapter 2 baseline grew.

v1

Initial release: four controls, requirements typed by hand, catalog anchored to `main`, no graph verification.